



Faculty of Engineering and Technology
Electrical and Computer Engineering Department
Computer Networks – ENCS3320

Project #1

Socket Programming

Prepared by:

Name: Mustafa Altaweel

UID: 1221017

Abstract

This project aims to help students to gain a better understanding of socket programming and computer networking, by providing practical experience with fundamental key network commands, and focusing on understanding, implementation and creation of two important protocols; Transmission control protocol (TCP), and User datagram protocol (UDP) sockets.

Contents

Abstract.....	2
Table of Contents:	Error! Bookmark not defined.
Table of Figures:	Error! Bookmark not defined.
Theory:	5
Socket programming basics:	5
Key Features:.....	5
Key Features:.....	6
Methodology and Tools:.....	7
Python:.....	7
a. Server Side:	8
b. Client Side:.....	11
C. Run Example:	13
Alternative Solutions, Issues, and Limitations:	18
Alternative Solutions:	18
Issues, and Limitations:	18
References:	19

Table of Figures

Figure 1 TCP Diagram	5
Figure 2 UDP Diagram.....	6
Figure 3 Comparison diagram between TCP and UDP.....	6
Figure 4 server information	8
Figure 5 socket object initiate.....	8
Figure 6 Client handling function.....	9
Figure 7 Send question to all players function	10
Figure 8 End game function	10
Figure 9 Collect answers function.....	11
Figure 10 Send scoreboard function.....	11
Figure 11 Client configurations.....	12
Figure 12 Client connection function	12
Figure 13 Main client function	13
Figure 14 Example run (1).	14
Figure 15 Example run (2).	15
Figure 16 Example run (3).	16
Figure 17 Example run (4).	17

Theory:

Socket programming basics:

Socket programming is the foundation of network communication between devices, enabling data exchange over a network. Sockets provide an endpoint for sending or receiving data and support two key communication protocols: **Transmission Control Protocol (TCP)** and **User Datagram Protocol (UDP)**.

TCP (Transmission Control Protocol):

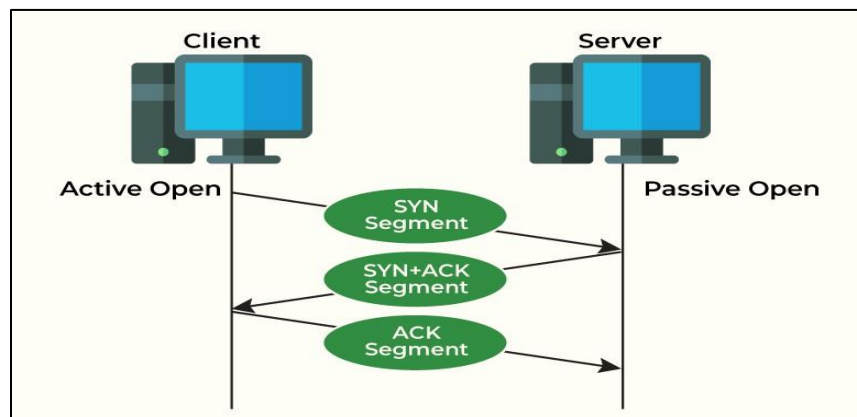


Figure 1 TCP Diagram

TCP is a connection-oriented protocol, which means it establishes a reliable communication channel between the client and server before transmitting data. This ensures data integrity, as packets are delivered in the correct order and without loss. TCP is commonly used in applications like web browsing and email.

Key Features:

- 1- Reliable and error-checked data delivery.
- 2- Stream-oriented communication.
- 3- Slower than UDP due to overhead for reliability.

UDP (User Datagram Protocol):

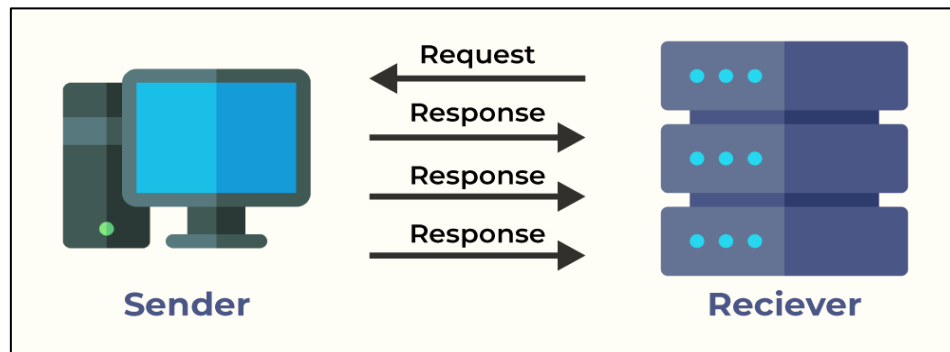


Figure 2 UDP Diagram.

UDP is a connectionless protocol, which means data is sent without establishing a connection. It is faster than TCP but does not guarantee delivery or order of packets. UDP is commonly used in applications requiring low latency, such as live video streaming and online gaming.

Key Features:

- 1- Faster but less reliable.
- 2- Packet-oriented communication.
- 3- Suitable for time-sensitive applications.

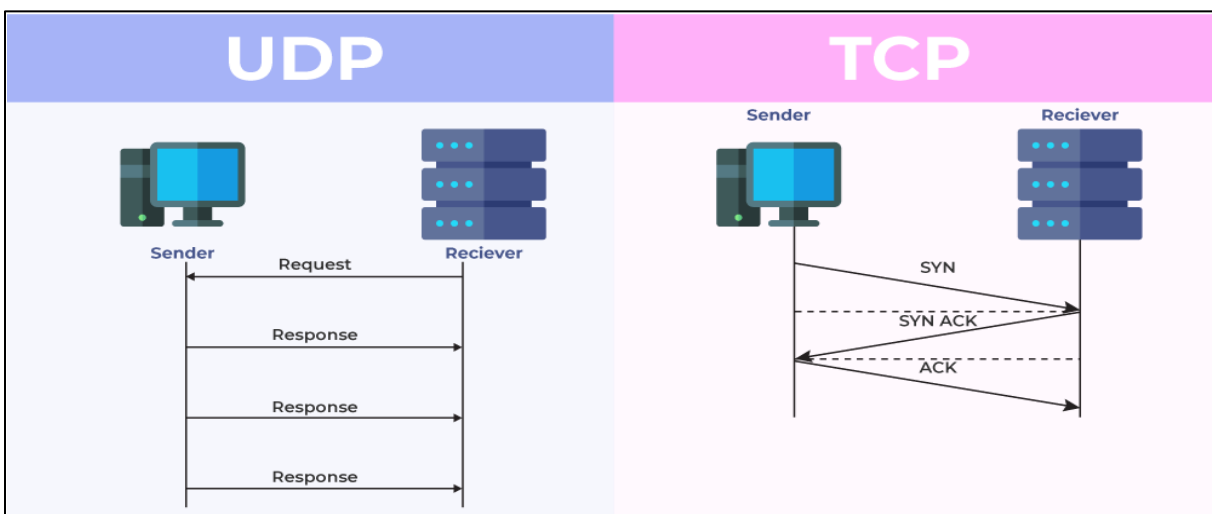


Figure 3 Comparison diagram between TCP and UDP.

Methodology and Tools:

Programming Languages:

To implement socket programming, various programming languages like **Python** and **C** are commonly used.

Python:

Python is widely favored for its simplicity and extensive library support. The built-in socket library makes it easy to create TCP and UDP sockets with minimal code, we used python because it has a **High-level abstraction**, and a **Fast development cycle**

UDP Server Client Game:

We are required to make a game that contains server and client side using UDP (User Datagram Protocol), we decided to make the program using python language due to its simplicity and functionality.

a. Server Side:

In server side, we started our code by some configurations, server IP, port number, and buffer size.

```
1 import socket
2 import random
3 import time
4
5 PORT = 5689 # The port number that will be used according to the project specifications.
6 SERVER = socket.gethostname(socket.gethostname()) # The IP of the host that is using the server.
7 ADDR = (SERVER, PORT) # The data that will be written on the packet to route it correctly(IP, and Port used).
8 FORMAT = "utf-8" # Encoding format since the messages will be sent in binary format.
9 DIS_MESSAGE = "." # The sent message in case of disconnecting from the server.
10 BUFFER_SIZE = 1024 # Maximum message size.
```

Figure 4 server information

We made a socket object with **datagram** type to confirm that we are using UDP, which will handle the connections with any client willing to join.

```
server_connect = socket.socket(socket.AF_INET, socket.SOCK_DGRAM) # Creates a new socket object with
server_connect.bind(ADDR) # Associates the socket with the specified address
```

Figure 5 socket object initiate

After server initialization, we made the game functions, and the first one that will handle players and take their IP's and names, and notify them about game or other clients updates, which called *handle_client()*.


```

43     server_connect.sendto(server_msg.encode(FORMAT), client) # notify all clients that the game started
44     time.sleep(3)
45     print(f"[START] The game started!")
46     start_game() # Start game
47     continue
48
49     username = clients[address]
50     print(f"[{username} - {address}] {msg}")
51
52     if msg == DIS_MESSAGE: # If the client send the disconnection message
53         print(f"[DISCONNECTION] {username} at {address} disconnected.")
54         del clients[address]
55         del scores[address]
56         connections -= 1
57         continue
58
59     server_msg = "Message received".encode(FORMAT)
60     server_connect.sendto(server_msg, address)
61

```

Figure 6 Client handling function

Other function we made is called [*start_game\(\)*](#), which purpose is to randomly pick number of rounds and questions each round, and uses another function, [*send_question_to_all_clients\(\)*](#), to send the questions updates to all players, and when number of rounds is exceeded, we call [*end_game\(\)*](#) function to notify all players that the game has ended.

```
def start_game():
    for round_num in range(1, 4): # Randomly choose number of rounds
        print(f"[ROUND {round_num}] Starting round {round_num}...")
        for client in clients:
            server_connect.sendto(f"[ROUND {round_num}] Get ready!".encode(FORMAT), client) # Notify all clients the round number

        num_questions = random.randint(2, 4) # Randomly choose the number of questions for the current round
        for _ in range(num_questions):
            send_question_to_all_clients() # Send the Question to all clients

        print(f"[ROUND {round_num}] Round {round_num} completed!")

    end_game() # End the game after rounds finishes
```

```
def send_question_to_all_clients():

def end_game():
    print("[GAME OVER] Ending the game.")
    for client in clients:
        server_connect.sendto("game over".encode(FORMAT), client)
    print("[FINAL SCORES]")
    for client, score in scores.items():
        print(f"[client={client}] {score}")
```

Figure 7 Send question to all players function

Figure 8 End game function

Helper methods that are used for server side are [*collect_answers\(\)*](#) that collects all clients answers and see if they answered correctly. Also, [*send_scoreboard\(\)*](#) that broadcasts each question the players scoreboard to all players.

```

7 def collect_answers(question):
8     start_time = time.time()
9     received_answers = {}
10
11     while time.time() - start_time < 15: # Set timer of 15s for each round
12         try:
13             server_connect.settimeout(15 - (time.time() - start_time))
14             answer_msg, client = server_connect.recvfrom(BUFFER_SIZE) # Collect answers from clients
15             answer_msg = answer_msg.decode(FORMAT)
16
17             if client not in clients:
18                 continue
19
20             if client not in received_answers:
21                 received_answers[client] = answer_msg
22         except socket.timeout:
23             break
24
25     for client, answer in received_answers.items():
26         if answer.lower() == question["answer"].lower(): # If answer is correct
27             scores[client] += 1
28             print(f"[CORRECT ANSWER] {clients[client]} answered correctly! New score: {scores[client]}")
29             server_connect.sendto(f"You answered correctly! Your score now is: {scores[client]}".encode(FORMAT), client)
30         else: # If answer is wrong
31             print(f"[WRONG ANSWER] {clients[client]} answered incorrectly.")
32             server_connect.sendto(f"You answered wrong! Your score still: {scores[client]}".encode(FORMAT), client)

```

Figure 9 Collect answers function

```

def send_scoreboard():
    scoreboard = "[SCOREBOARD]\n" + "\n".join([f"{clients[client]}: {scores[client]}" for client in clients])
    print(scoreboard)
    for client in clients:
        server_connect.sendto(scoreboard.encode(FORMAT), client)

```

Figure 10 Send scoreboard function

b. Client Side:

We treated the clients as the players of the game, which we need their IPs and names, so we started with some configurations as in server side. And made a socket object as in server side using **datagram** to successfully work on UDP protocol.

```

1  import socket
2
3  PORT = 5689
4  SERVER = socket.gethostname([socket.gethostname()])
5  ADDR = (SERVER, PORT)
6  FORMAT = "utf-8"
7  DIS_MESSAGE = "."
8  BUFFER_SIZE = 1024
9
10 client_connect = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
11
12 def send(msg):

```

Figure 11 Client configurations

We started the connection using function called `server_connection()`, which is responsible for taking the client's name and IP, then check the game status and broadcast result to the player.

```

def server_connection():
    client_name = input("Please enter your username: ")
    client_connect.sendto(client_name.encode(FORMAT), ADDR)
    print("Connected to the server.")

    while True:
        server_msg, _ = client_connect.recvfrom(BUFFER_SIZE)
        message = server_msg.decode(FORMAT)

        if message == "start":
            print("The game will start soon, be prepared!")
            break
        elif "joined the game" in message:
            print(f"[INFO]: {message}")
        elif message == "wait":
            print("Please wait for other players to join...")

    return True

```

Figure 12 Client connection function

We also the function, `send(msg)`, that is responsible for sending messages to the server, like name or question answers, and if there is a timeout the message won't be sent.

```

def send(msg):
    try:
        if msg == DIS_MESSAGE:
            print("Disconnecting from server...")
            client_connect.sendto(DIS_MESSAGE.encode(FORMAT), ADDR)
            client_connect.close()
            return False
        else:
            message = msg.encode(FORMAT)
            client_connect.sendto(message, ADDR)
            server_msg, _ = client_connect.recvfrom(BUFFER_SIZE)
            print(f"[SERVER]: {server_msg.decode(FORMAT)}")
            return True
    except socket.timeout:
        print("[ERROR]: No response from server, disconnecting...")
        return False

```

We reassembled all the functions above to one function called `main()`, which receive messages from the server and acts depending on the message info.

```

48 def main():
49     if server_connection():
50         while True:
51             try:
52                 server_msg, _ = client_connect.recvfrom(BUFFER_SIZE)
53                 message = server_msg.decode(FORMAT)
54
55                 if message == "game over":
56                     print("The game has ended. Thank you for playing!")
57                     break
58                 elif "[QUESTION]" in message:
59                     print(f"{message}")
60                     answer = input("Your answer: ")
61                     if not send(answer):
62                         break
63                 elif "[SCOREBOARD]" in message:
64                     print(f"{message}")
65                 else:
66                     print(f"[INFO]: {message}")
67             except socket.timeout:
68                 print("[ERROR]: Server response timeout. Exiting game.")
69                 break
70

```

Figure 13 Main client function

C. Run Example:

Two Clients connected using **Hamachi** which allows you to setup a (VPN) Virtual Private Network that will allow you to share data between the computers in your network:

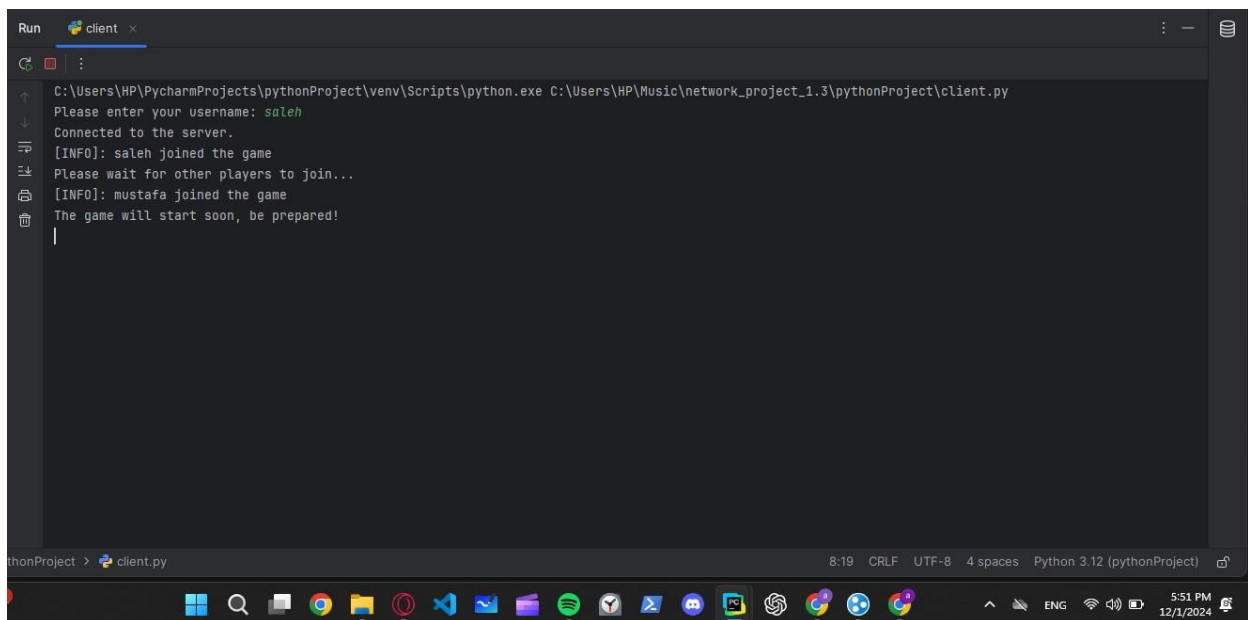
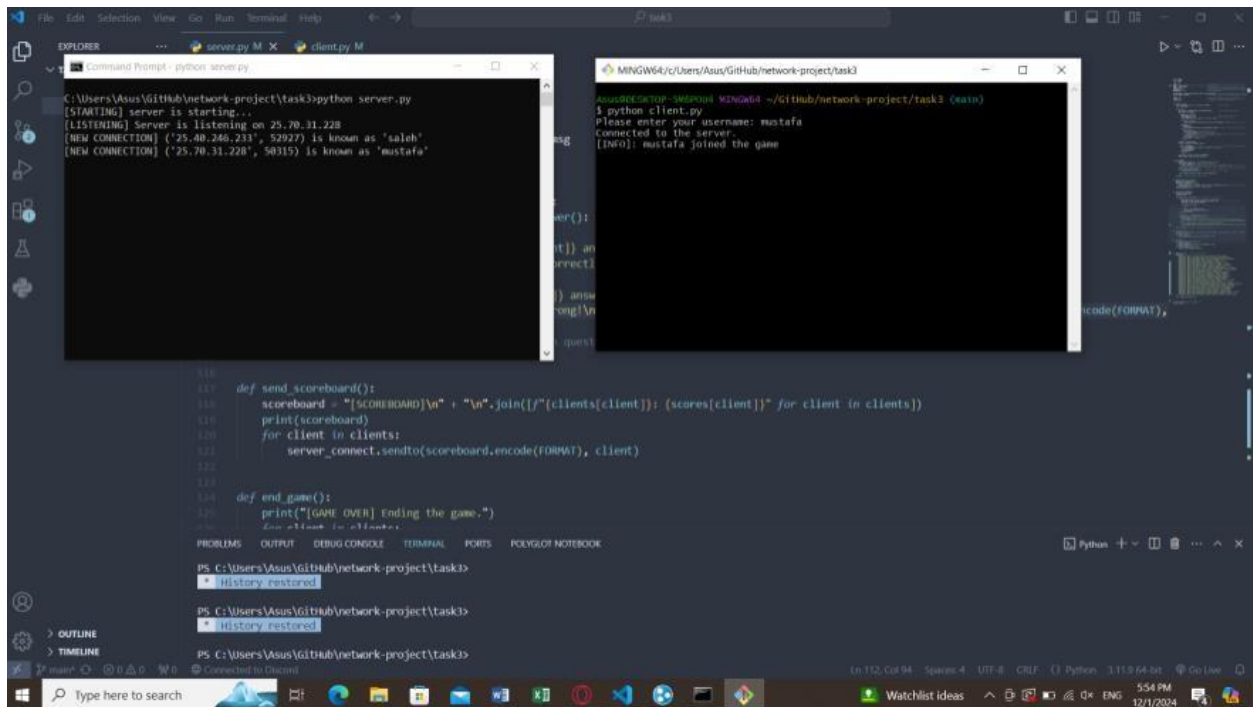


Figure 15 Example run (2).

The image shows a VS Code editor with a Python script and its execution in a terminal. The script is a multi-player trivia game. The terminal shows the game in progress with questions, answers, and a scoreboard.

```

118
119
120 def send_scoreboard():
121     scoreboard = "[SCOREBOARD]\n" + "\n".join([f"{clients[client]}: {scores[client]}" for client in clients])
122     print(scoreboard)
123     for client in clients:
124         server_connect.sendto(scoreboard.encode(FORMAT), client)
125
126 def end_game():
127     print("[GAME OVER] Ending the game.")
128
129
130
131
132
133
134
135
136
137
138
139
140
141
142
143
144
145
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160
161
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000

```

```

[QUESTION] sent to all players: Which planet is known as the Red Planet?
[WRONG ANSWER] mustafa answered incorrectly.
[WRONG ANSWER] saleh answered incorrectly.
[SCOREBOARD]
saleh: 2
mustafa: 2
[QUESTION] sent to all players: Who was the first human to step on the moon?
[WRONG ANSWER] saleh answered incorrectly.
[WRONG ANSWER] mustafa answered incorrectly.
[SCOREBOARD]
saleh: 2
mustafa: 2
[QUESTION] sent to all players: Who painted the Sistine Chapel ceiling?
[WRONG ANSWER] mustafa answered incorrectly.
[WRONG ANSWER] saleh answered incorrectly.
[SCOREBOARD]
saleh: 2
mustafa: 2
[QUESTION] sent to all players: Which country is known as the Land of the Rising Sun?
Your answer: japan
[SERVER]: You answered correctly! Your score now is: 3
[SCOREBOARD]
saleh: 3
mustafa: 3
[INFO]: [ROUND 3] Get ready!
[QUESTION] What is the hardest natural substance?
Your answer:

```

Figure 16 Example run (3).


```

[SCOREBOARD]
saleh: 3
mustafa: 4
[QUESTION] sent to all players: Who painted the Mona Lisa?
[CORRECT ANSWER] mustafa answered correctly! New score: 5
[CORRECT ANSWER] saleh answered correctly! New score: 4
[SCOREBOARD]
saleh: 4
mustafa: 5
[QUESTION] sent to all players: What is the freezing point of water in Celsius?
[WRONG ANSWER] saleh answered incorrectly.
[CORRECT ANSWER] mustafa answered correctly! New score: 6
[SCOREBOARD]
saleh: 4
mustafa: 6
[ROUND 3] Round 3 completed!
[GAME OVER] Ending the game.
[FINAL SCORES]
saleh: 4
mustafa: 6

def send_scoreboard():
    scoreboard = "[SCOREBOARD]\n" + "\n".join([f"[clients[{client}]]: {scores[client]}" for client in clients])
    print(scoreboard)
    for client in clients:
        server_connect.sendto(scoreboard.encode(FORMAT), client)

def end_game():
    print("[GAME OVER] Ending the game.")
    # Don't think to do this

PS C:\Users\Asus\Github\network-project\task3>
* History restored
PS C:\Users\Asus\Github\network-project\task3>
* History restored
PS C:\Users\Asus\Github\network-project\task3>

```

```

[QUESTION] What is the hardest natural substance?
Your answer: gold
[SERVER]: You answered wrong!
Correct answer is:diamond. Your score still: 3
[SCOREBOARD]
saleh: 3
mustafa: 4
[QUESTION] Who painted the Mona Lisa?
Your answer: da vinci
[SERVER]: You answered correctly! Your score now is: 4
[SCOREBOARD]
saleh: 4
mustafa: 5
[QUESTION] What is the freezing point of water in Celsius?
Your answer: 100
[SERVER]: You answered wrong!
Correct answer is:0. Your score still: 4
[SCOREBOARD]
saleh: 4
mustafa: 6
The game has ended. Thank you for playing!
(venv) PS C:\Users\HP\Music\network_project_1.3\pythonProject>

```

Figure 17 Example run (4).

Alternative Solutions, Issues, and Limitations:

Alternative Solutions:

C/C++ Programming:

In this project we used python for lots of reason mentioned in the theory part, we could have used C/C++, they offer lower-level control over system resources, which can be advantageous for performance-critical applications. By using the socket API in C, developers have direct access to memory management and can optimize network communication for speed. However, this comes at the cost of increased complexity and potential memory management errors. The reason why we didn't use C or C++ is that it is **much harder** and the **memory control** is **manual** which can introduce many **bugs** and **memory leaks**.

Issues, and Limitations:

Error Handling and Exception Management:

One of the challenges faced while using Python's socket library is handling network errors gracefully. Sockets in Python, especially when dealing with unreliable protocols like UDP, can experience packet loss or timeouts. While TCP guarantees packet delivery, UDP does not, which can result in data loss or the need for additional error-checking mechanisms.

References:

- 1- <https://davisgitonga.dev/blog/request-response-cycle>
- 2- <https://www.geeksforgeeks.org/differences-between-tcp-and-udp/>
- 3- Computer network a top down approach 8TH edition by Jim Kurose, Keith Ross